CHAPTER 2

A Simple Compiler

To provide an overview of how the compilation process can be organized, we will consider in some detail how a compiler can be built for a very small programming language. Our language is called *Micro*. It is an extremely simple language, lacking even enough features to write a useful program. Micro is designed only to provide a concrete language around which we can discuss a simple example compiler.

We first define Micro informally:

- The only data type is integer.
- All identifiers are implicitly declared and are no longer than 32 characters. Identifiers must begin with a letter and are composed of letters, digits, and underscores.
- Literals are strings of digits.
- Comments begin with -- and end at the end of the current line.

- Statement types are

  Assignment:

  ID := Expression ;

  Expression is an infix expression constructed from identifiers, literals, and the operators + and −; parentheses are also allowed.

  Input/Output:

  read (List of IDs) ;
  write (List of Expressions) ;

- **begin**, **end**, **read**, and **write** are reserved words.
- Each statement is terminated by a semicolon (;). The body of a program is delimited by **begin** and **end**.
- A blank is appended to the right end of each source line; thus tokens may not extend across line boundaries.

2.1. The Structure of a Micro Compiler

A simple compiler for Micro is the subject of the rest of this chapter. The structure of this compiler is based on that illustrated in Figure 1.3. In the interest of simplicity, the compiler will be a one-pass type, whose most important feature is that no explicit intermediate representations are used. The interfaces between the components will be as follows:

- The scanner reads a source program from a text file and produces a stream of token representations (these will be defined more precisely in Chapter 3). So that no actual stream need exist at any time, the scanner is actually a function that produces token representations one at a time when called by the parser.
- The parser processes tokens until it recognizes a syntactic structure that requires semantic processing. It then makes a direct call to a semantic routine. Some of these semantic routines use token representation information in their processing.
- The semantic routines produce output in assembly language for a simple three-address virtual machine. Thus the compiler structure includes no optimizer, and code generation is done by direct calls to appropriate support routines from the semantic routines.
- The symbol table is used only by the semantic routines. Its interface is described in Section 2.5.5.

2.2. A Micro Scanner

The definition of Micro tokens can be formalized, as we shall see in Chapter 3. For our present purposes an informal definition will suffice. The first step in building a Micro compiler is to construct a Micro scanner. We will define an enumeration type **token** that will represent the set of Micro tokens. Our scanner will be a function of no arguments that returns **token** values.

```
typedef enum token_types {
    BEGIN, END, READ, WRITE, ID, INTLITERAL,
    LPAREN, RPAREN, SEMICOLON, COMMA, ASSIGNOP,
    PLUSOP, MINUSOP, SCANEOF
} token;

extern token scanner(void);
```

The scanner will read characters and group them into tokens. Care is required that we don't sometimes read too much. In particular, we may need to see the beginning of the next token in order to recognize the end of the current token. For Micro, all that is ever needed is one character of lookahead. The extra character can be conveniently "pushed back" onto the input using the standard **ungetc()** function.

For simplicity, we will assume that input is coming from **stdin**; in practice a source file would have to be opened and an explicit **FILE** pointer would be used.

When called, the scanner must find the beginning of some token. To do this, it first inspects the next input character. If the character cannot begin any token, we have a *lexical* or *token* error. An error message is issued, and we then attempt to *recover* from the error. A simple way to do this is to skip the character and restart scanning. This process continues until the beginning of some token is found. Then we match the longest possible character sequence that comprises a legal token.

Figure 2.1 shows code for the main loop of a scanner that can recognize Micro identifiers and literals (integer constants). It also skips white space (blanks, tabs, and end-of-line markers). For simplicity, we assume an "end-of-line" character exists. In C, this is usually called a "newline," and is denoted by the *escape sequence* **'\n'**. Even if our character set lacks a specific "newline" character, the C I/O library will return **'\n'** when some sort of end of line marker has been seen.

Operators, comments, and delimiters are easy to add. Figure 2.2 shows code that has been added to the loop in Figure 2.1 for recognizing these tokens.

We have not yet included recognition of reserved words in the Micro scanner. The problem, of course, is that reserved words look the same as identifiers. We might require that reserved words be somehow specially marked (for example, by putting them in quotes or in uppercase), but this

```
#include <stdio.h>
#include <ctype.h>

int in_char, c;

while ((in_char = getchar()) != EOF) {
    if (isspace(in_char))
        continue;     /* do nothing */
    else if (isalpha(in_char)) {
        /*
         * ID ::= LETTER | ID LETTER
         *               | ID DIGIT
         *               | ID UNDERSCORE
         */
        for (c = getchar(); isalnum(c) || c == '_';
                c = getchar())
            ;
        ungetc(c, stdin);

        return ID;
    } else if (isdigit(in_char)) {
        /*
         * INTLITERAL ::= DIGIT |
         *                INTLITERAL DIGIT
         */
        while (isdigit((c = getchar())))
            ;
        ungetc(c, stdin);

        return INTLITERAL;
    } else
        lexical_error(in_char);
}
```

Figure 2.1 Scanner Loop to Recognize Identifiers and Integer Literals

approach is a nuisance to programmers, so we would rather avoid it. Two other approaches are commonly used to differentiate between identifiers and reserved words. In the first, the scanner has a table of reserved words that is checked whenever an identifier is recognized. If a token is on this list, then it is always interpreted as a reserved word rather than as an identifier. Alternately, reserved words can be entered in the symbol table as part of the initialization of the compiler with a special attribute, *reserved*. After an identifier is recognized by the scanner, it is looked up in the symbol table. If it is found and has this special attribute, it is recognized as a reserved word.

For Micro, either approach is workable. We'll assume the scanner has a routine `check_reserved()` that takes the identifiers as they are recognized and returns the proper token class (either `ID` or some reserved word).

```
#include <stdio.h>
#include <ctype.h>

int in_char, c;

while ((in_char = getchar()) != EOF) {
    if (isspace(in_char))
        /* do nothing */
        continue;
    else if (isalpha(in_char))
        /* code to recognize identifiers goes here */
    else if (isdigit(in_char))
        /* code to recognize int literals goes here */
    else if (in_char == '(')
        return LPAREN;
    else if (in_char == ')')
        return RPAREN;
    else if (in_char == ';')
        return SEMICOLON;
    else if (in_char == ',')
        return COMMA;
    else if (in_char == '+')
        return PLUSOP;
    else if (in_char == ':') {
        /* looking for ":=" */
        c = getchar();
        if (c == '=')
            return ASSIGNOP;
        else {
            ungetc(c, stdin);
            lexical_error(in_char);
        }
    } else if (in_char == '-') {
        /* looking for --, comment start */
        c = getchar();
        if (c == '-') {
            while ((in_char = getchar()) != '\n');
        } else {
            ungetc(c, stdin);
            return MINUSOP;
        }
    } else
        lexical_error(in_char);
}
```

Figure 2.2 Scanner Loop with New Code to Recognize Operators, Comments, and Delimiters

However, the scanner code for identifier recognition in Figure 2.1 is inadequate for discovering reserved words this way. We haven't made any provision for saving the characters of a token as they are scanned. For very simple tokens, like operators or delimiters, knowing the token class suffices. For others, such as identifiers and literals, we need the actual text of the token. To do this, we will call a routine `buffer_char()` that adds its argument to a character buffer called `token_buffer`. `clear_buffer()` will reset the buffer to the empty string. This buffer is visible to any part of the compiler and always contains the text of the most recently scanned token. In our example compiler, we will be particularly interested in use of `token_buffer` by semantic routines. The characters in this buffer also will be used by `check_reserved()` to determine whether a token that looks like an identifier is actually a reserved word.

We also must decide how to handle end of file. The parser must know when the input is exhausted in order to verify that a complete program has been parsed, so we have created an end-of-file token called `SCANEOF`. The token is often denoted by $ in formal descriptions of parsing algorithms and by parser generators. However, this is not a valid enumeration literal in any typical programming language, so we use `SCANEOF` instead. Whenever the scanner is called with `feof(stdin)` true, it will return `SCANEOF`.

Figure 2.3 contains the complete code for the main routine of the scanner. The auxiliary routines used by this routine (`buffer_char()`, and so on) are not included.

```
#include <stdio.h>
/* character classification macros */
#include <ctype.h>

extern char token_buffer[];

token scanner(void)
{
    int in_char, c;

    clear_buffer();
    if (feof(stdin))
        return SCANEOF;

    while ((in_char = getchar()) != EOF) {
        if (isspace(in_char))
            continue;     /* do nothing */
        else if (isalpha(in_char)) {
            /*
             * ID ::= LETTER | ID LETTER
             *               | ID DIGIT
             *               | ID UNDERSCORE
             */
            buffer_char(in_char);
            for (c = getchar(); isalnum(c) || c == '_';
                    c = getchar())
                buffer_char(c);
```

```
            ungetc(c, stdin);
            return check_reserved();
        } else if (isdigit(in_char)) {
            /*
             * INTLITERAL ::= DIGIT |
             *                INTLITERAL DIGIT
             */
            buffer_char(in_char);
            for (c = getchar(); isdigit(c);
                    c = getchar())
                buffer_char(c);
            ungetc(c, stdin);
            return INTLITERAL;
        } else if (in_char == '(')
            return LPAREN;
        else if (in_char == ')')
            return RPAREN;
        else if (in_char == ';')
            return SEMICOLON;
        else if (in_char == ',')
            return COMMA;
        else if (in_char == '+')
            return PLUSOP;
        else if (in_char == ':') {
            /* looking for ":=" */
            c = getchar();
            if (c == '=')
                return ASSIGNOP;
            else {
                ungetc(c, stdin);
                lexical_error(in_char);
            }
        } else if (in_char == '-') {
            /* is it --, comment start */
            c = getchar();
            if (c == '-') {
                do
                    in_char = getchar();
                while (in_char != '\n');
            } else {
                ungetc(c, stdin);
                return MINUSOP;
            }
        } else
            lexical_error(in_char);
    }
}
```

Figure 2.3 Complete Scanner Function for Micro

2.3. The Syntax of Micro

Rather than defining the syntax of Micro informally, we will give a precise definition using a context-free grammar (CFG). A CFG is also sometimes called a BNF (Backus-Naur form) grammar.

Informally, a CFG is simply a set of rewriting rules or *productions*. A production is of the form

$$A \rightarrow B\ C\ D\ \cdots\ Z$$

A is the *left-hand side* (LHS) of the production; B C D $\cdots$ Z constitute the *right-hand side* (RHS) of the production. Every production has exactly one symbol on its LHS; it can have any number of symbols (zero or more) on its RHS. A production represents the rule that any occurrence of its LHS symbol can be replaced by the symbols on its RHS. Thus the production

<program> → **begin** <statement list> **end**

states that a program is required to be a statement list delimited by a **begin** and **end**.

Two kinds of symbols may appear in a CFG: *nonterminals* and *terminals*. In this text, nonterminals are often delimited by < and > for ease of recognition. However, nonterminals can also be recognized by the fact that they appear on the left-hand sides of productions. A nonterminal is, in effect, a placeholder. All nonterminals must be replaced, or rewritten, by a production having the appropriate nonterminal on its LHS. In contrast, terminals are never changed or rewritten. Rather, they represent the *tokens* of a language. Thus the overall purpose of a set of productions (a CFG) is to specify what sequences of terminals (tokens) are legal. A CFG does this in a remarkably elegant way: We start with a single nonterminal symbol called the *start* or *goal* symbol. We then apply productions, rewriting nonterminals until only terminals remain. Any sequence of terminals that can be produced by doing this is considered legal. Similarly, if a sequence of terminals *cannot* be produced by any sequence of nonterminal replacements, then that sequence is deemed illegal. To see how this works, let us look at a CFG for Micro. λ will represent the empty or null string. Thus a production $A \rightarrow \lambda$ states that A can be replaced by the empty string, effectively erasing it.

Programming language constructs often involve optional items, or lists of items. To cleanly represent such features, an *extended BNF* notation is often utilized. An optional item sequence is enclosed in square brackets, [and]. For example, in

<program> → [ID :] **begin** <statement list> **end**

a program can be optionally labeled. Optional lists are enclosed by braces, { and }. Thus in

<statement list> → <statement> {<statement>}

a statement list is defined to be a single statement, optionally followed by *zero or more* additional statements.

An extended BNF has the same definitional capability as ordinary BNFs. In particular, the following transforms can be used to map extended BNFs into standard form. An optional sequence is replaced by a new nonterminal that generates λ or the items in the sequence. Similarly, an optional list is replaced by a new nonterminal that generates λ or the list items *followed by* the nonterminal. Thus our statement list can be transformed into

<statement list> → <statement> <statement tail>
<statement tail> → λ
<statement tail> → <statement> <statement tail>

The advantage of extended BNFs is that they are more compact and readable. We can envision a preprocessor that takes extended BNFs and produces standard BNFs, using these transforms.

Figure 2.4 shows an extended CFG for Micro. An *augmenting* production

<system goal> → <program> SCANEOF

appears in the grammar to make sure that the string matched by <system goal> includes all the input. It specifies that the end-of-file marker, SCANEOF, *must follow* after the last valid token of a program.

1. <program> → **begin** <statement list> **end**
2. <statement list> → <statement> {<statement>}
3. <statement> → ID := <expression> ;
4. <statement> → **read** (<id list>) ;
5. <statement> → **write** (<expr list>) ;
6. <id list> → ID {, ID}
7. <expr list> → <expression> {, <expression>}
8. <expression> → <primary> {<add op> <primary>}
9. <primary> → (<expression>)
10. <primary> → ID
11. <primary> → INTLITERAL
12. <add op> → PLUSOP
13. <add op> → MINUSOP
14. <system goal> → <program> SCANEOF

Figure 2.4 Extended CFG Defining Micro

To see how this grammar defines legal Micro programs, let's follow the *derivation* of one such program, **begin** ID := ID + (INTLITERAL – ID) ; **end**, starting from the nonterminal <program>.

<program>	
begin <statement list> **end**	(Apply rule 1)
begin <statement> {<statement>} **end**	(Apply rule 2)
begin <statement> **end**	(Choose 0 repetitions)
begin ID := <expression> ; **end**	(Apply rule 3)
begin ID := <primary> {<add op> <primary>} ; **end**	(Apply rule 8)
begin ID := <primary> <add op> <primary> ; **end**	(Choose 1 repetition)
begin ID := <primary> + <primary> ; **end**	(Apply rule 12)
begin ID := ID + <primary> ; **end**	(Apply rule 10)
begin ID := ID + (<expression>) ; **end**	(Apply rule 9)
begin ID := ID + (<primary> {<add op> <primary>}) ; **end**	(Apply rule 8)
begin ID := ID + (<primary> <add op> <primary>) ; **end**	(Choose 1 repetition)
begin ID := ID + (<primary> – <primary>) ; **end**	(Apply rule 13)
begin ID := ID + (INTLITERAL – <primary>) ; **end**	(Apply rule 11)
begin ID := ID + (INTLITERAL – ID) ; **end**	(Apply rule 10)

At this point no nonterminals remain, so our derivation of a Micro program is completed.

A CFG defines a *language*, which is a set of sequences of tokens. Any sequence of tokens that can be derived using the grammar is valid; any sequence that cannot be derived is not valid. Actually, to be precise, any token sequence derivable from a CFG is considered syntactically valid. When static semantics are checked by the semantic routines, *semantic errors* in a syntactically valid program may be discovered. For example, in Pascal the statement

```
A := 'X' + True;
```

has no syntax errors but it does have a semantic error: the operator + is not defined for adding a character to a boolean value.

Structure as well as syntax can be defined in a CFG. For expressions, this includes *associativity* and *operator precedence*. Associativity is concerned with the order in which consecutive instances of an operator are applied (as in A–B–C). Operator precedence refers to the relative priority of operators. For example, we expect A+B*C to mean A+(B*C), since * is usually considered to be a higher precedence operator than +. Micro has only one level of precedence, because it does not include multiplication. If multiplication were included, it would be at a higher level of precedence than addition. The following grammar fragment defines such a precedence relationship.

```
<expression> → <factor> {<add op> <factor>}
<factor>     → <primary> {<mult op> <primary>}
<primary>    → ( <expression> )
<primary>    → ID
<primary>    → INTLITERAL
```

Examining a *derivation tree* for the expression A+B*C, which is derived from this grammar fragment, illustrates how this definition works. A deriva-

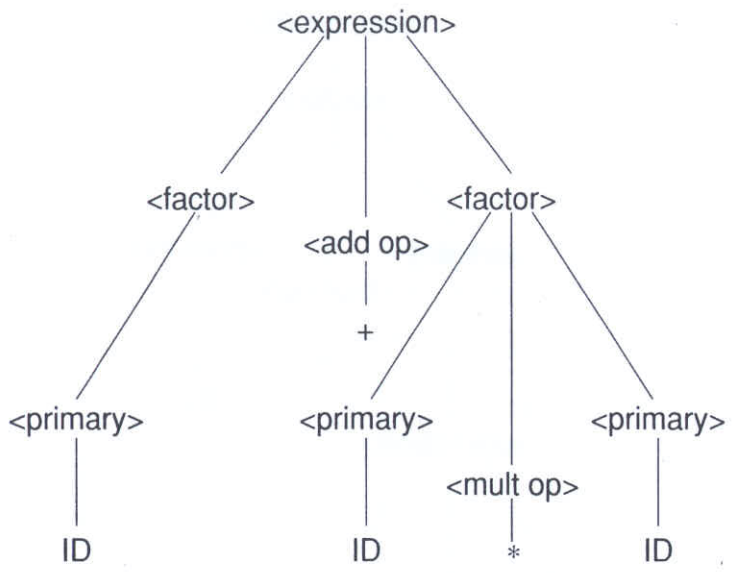

Figure 2.5 Derivation Tree for A+B*C

tion tree is formed by showing nonterminal expansions as subtrees. Figure 2.5 shows the tree for this expression. The tree shows that * has a higher precedence than + because the second and third IDs are grouped together (with *) in a subtree, and *then* this subtree is grouped together with the first ID (using +) in the main derivation tree.

Can a derivation tree with *wrong* precedence ever be formed in this grammar? No—the production rules don't allow it. Try to build ID+ID*ID with + applied first. Since a * can only be generated by a <factor>, ID+ID must appear in a subtree rooted by <primary>. However, <primary> *cannot* generate ID+ID unless the subexpression is enclosed in parentheses. With parentheses, the desired grouping can be forced, as illustrated in Figure 2.6.

2.4. Recursive Descent Parsing

For our Micro parser, we will use a well-known parsing technique called *recursive descent*. Its name is taken from the recursive parsing routines that, in effect, descend through the parse tree it recognizes as it processes a program. Recursive descent is one of the simplest parsing techniques used in practical compilers. The basic idea of recursive descent parsing is that each nonterminal has an associated *parsing procedure* that can recognize any sequence of tokens generated by that nonterminal. Within a parsing procedure, both nonterminals and terminals can be matched. To match a nonterminal A, we call the parsing procedure corresponding to A which, by convention, is named `A`. These calls may be recursive, hence the name recursive descent. To match a terminal symbol `t`, we call a procedure `match(t)`. `match()` calls the scanner to get the next token. If it is `t`, everything is correct and the token is

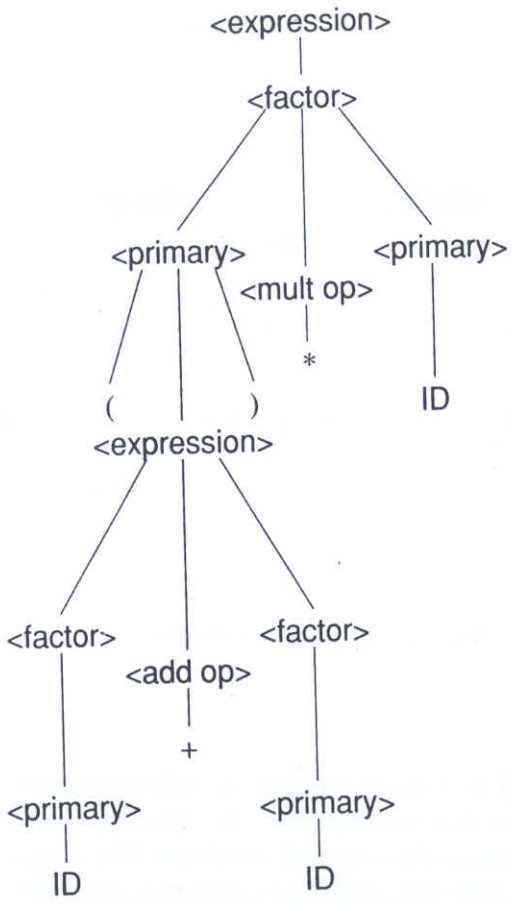

Figure 2.6 Derivation Tree for (A+B)*C

saved in a global variable named **current_token**. If this token is not **t**, we have found a *syntax error*, and an error message is produced. Some error correction or repair is then done to restart the parser and continue compilation.

To see how this works, consider the parsing routines that would be written to correspond to the productions of the Micro grammar. The parser is started by calling the procedure corresponding to <system goal>:

```
void system_goal(void)
{
    /* <system goal> ::= <program> SCANEOF */
    program();
    match(SCANEOF);
}
```

That is, to correctly parse a Micro program, we must match a token sequence generated by <program> followed by **SCANEOF**. Similarly for <program>, we have the following procedure:

```
void program(void)
{
    /* <program> ::= BEGIN <statement list> END */
    match(BEGIN);
    statement_list();
    match(END);
}
```

For <statement list>, we must decide how to deal with the optional statement sequence. Let `next_token()` be a function that returns the next token to be matched. If `next_token()` can be generated as the first (that is, leftmost) token from the nonterminal <statement>, we will try to recognize an optional statement. Otherwise, we will conclude that an entire statement list has been matched. This approach does not work for all CFGs, but it does for LL(1) grammars, a subset of CFGs that is well suited to recursive descent parsing. LL(1) grammars and parsing will be discussed in detail in Chapter 5.

How do we decide which tokens can be derived as the first tokens of a nonterminal? In Chapter 5 we show how to automatically compute these tokens, but for a CFG as small as Micro, this can be done by inspection. Assume we want the first tokens for a nonterminal A. Denote this token set as First(A). We select all productions with A as the left-hand side. For each such production, we simply examine the leftmost symbol in its right-hand side. If that symbol is a terminal, we include it in First(A). If the symbol is a nonterminal, B, we compute First(B), and include it in First(A). In the case of <statement>, things are simple because all <statement> productions start with terminals. The appropriate parsing procedure is as follows:

```
void statement_list(void)
{
    /*
     * <statement list> ::= <statement>
     *                      { <statement> }
     */
    statement();
    while (TRUE)  {
        switch (next_token()) {
        case ID:
        case READ:
        case WRITE:
            statement();
            break;
        default:
            return;
        }
    }
}
```

In defining the parsing procedure corresponding to <statement> we run into the problem that more than one production has <statement> as an LHS; thus we must decide which production to try to match. To do this, we must carefully analyze what each production can generate. We consider the First sets corresponding to each <statement> production. If these sets are unique for each production, we can make a unique choice—we pick the production that contains `next_token()` in its First set. If a production has λ as its RHS, we make it a default and match it whenever no other production is selected (obviously λ generates an empty First set). If `next_token()` appears in *no* First set and we have no λ production, we have a syntax error, because no production can match the next token.

Not all CFGs have the property that productions sharing the same LHS can be distinguished on the basis of their first token sets. However, LL(1) grammars *do* have this property, which is why they are so well suited to recursive descent parsing.

The parsing procedure for <statement> is shown in Figure 2.7, along with the rest of the parsing procedures for Micro, which are constructed using the techniques we have presented in this section.

```
void statement(void)
{
    token tok = next_token();

    switch (tok) {
    case ID:
        /* <statement> ::= ID := <expression> ; */
        match(ID); match(ASSIGNOP);
        expression(); match(SEMICOLON);
        break;

    case READ:
        /* <statement> ::= READ ( <id list> ) ; */
        match(READ); match(LPAREN);
        id_list(); match(RPAREN);
        match(SEMICOLON);
        break;

    case WRITE:
        /* <statement> ::= WRITE ( <expr list> ) ; */
        match(WRITE); match(LPAREN);
        expr_list(); match(RPAREN);
        match(SEMICOLON);
        break;

    default:
        syntax_error(tok);
        break;
    }
}
```

```
void id_list(void)
{
    /* <id list> ::= ID { , ID } */
    match(ID);

    while (next_token() == COMMA) {
        match(COMMA);
        match(ID);
    }
}

void expression(void)
{
    token t;

    /*
     * <expression> ::= <primary>
     *                  { <add op> <primary> }
     */
    primary();
    for (t = next_token(); t == PLUSOP || t == MINUSOP;
                t = next_token()) {
        add_op();
        primary();
    }
}

void expr_list(void)
{
 /* <expr list> ::= <expression> { , <expression> } */
    expression();

    while (next_token() == COMMA) {
        match(COMMA);
        expression();
    }
}

void add_op(void)
{
    token tok = next_token();

    /* <addop> ::= PLUSOP | MINUSOP */
    if (tok == PLUSOP || tok == MINUSOP)
        match(tok);
    else
        syntax_error(tok);
}
```

```
void primary(void)
{
    token tok = next_token();

    switch (tok) {
    case LPAREN:
        /* <primary> ::= ( <expression> ) */
        match(LPAREN); expression();
        match(RPAREN);
        break;

    case ID:
        /* <primary> ::= ID */
        match(ID);
        break;

    case INTLITERAL:
        /* <primary> ::= INTLITERAL */
        match(INTLITERAL);
        break;

    default:
        syntax_error(tok);
        break;
    }
}
```

Figure 2.7 Remaining Parsing Procedures for Micro

2.5. Translating Micro

2.5.1. Target Language

We are now ready to begin work on the actual translation of Micro. First, we must decide what machine we will generate code for and in what form (assembly code, object module, or whatever) the generated code will be produced. For simplicity, we will use assembly code for a three-address machine. Instructions for such a machine have the form

```
OP A,B,C
```

in which OP is an op-code (or pseudo-op), A and B designate operands of the specified operation, and C specifies the location where the result of the operation is to be stored. The operands may be variable names or integer literals. For some OPs, A or B or C may not be used (for example, a halt instruction is simply Halt). The format of our output will be character strings. For Micro, all arithmetic operations will be assumed to be done using integer arithmetic.

This target code is for a simple virtual machine; it could be used to drive an interpreter, or the individual instructions could be expanded by a more sophisticated code generator into code for a real machine. In fact, our target code bears a strong resemblance to *quadruples*, a commonly used intermediate representation. This point illustrates an interesting property of virtual instruction sets: They may be viewed as either an intermediate representation or as the output of a compiler, depending on how they are utilized.

2.5.2. Temporaries

During compilation, it is frequently necessary to use temporarily allocated storage locations, known as *temporaries*, to hold intermediate results of a computation. For our Micro compiler we will think of temporaries as *internal variables* that are implicitly declared when needed. This technique works well in Micro because variables are implicitly declared, too. Compilers for more realistic languages allocate temporaries from registers but may also use *storage temporaries* (memory locations, like those used in our Micro compiler) for special purposes—for example, when no registers are available or to save current register values before a procedure call. We will adopt a convention by which the internal variables used as temporaries are of the form Temp&N, where N is the index of the temporary, starting at 1. Since & cannot appear in an ordinary Micro variable, no conflict between variables and temporaries can arise.

2.5.3. Action Symbols

As explained in Chapter 1, the bulk of a translation is done by semantic routines called by the parser. Exactly when a given semantic routine is to be called is up to the compiler writer. *Action symbols* can be added to a grammar to specify when semantic processing should take place. Action symbols, denoted by #name in the examples that follow, can be placed anywhere in the RHS of a production. Corresponding to each action symbol is a semantic routine. Thus action symbol #add corresponds to a semantic routine named `add()`. When parsing procedures are created, calls to semantic routines or in-line code segments to do semantic processing are inserted in the positions designated by action symbols. If a grammar containing action symbols is given as input to a parser generator, the generator must include appropriate information in the tables it produces to trigger calls to semantic routines at corresponding times during the parsing process.

Action symbols have no impact on the language recognized by a parser driven by a CFG. Thus, they are not actually part of the syntax the CFG specifies. In this context, action symbols serve to "comment" a CFG, indicating when semantic actions need to be executed. When a CFG is used as the input to a parser generator, the action symbols are more than a comment. They tell the parser generator when the corresponding semantic routines must be called.

2.5.4. Semantic Information

An important issue in the design of semantic routines is the specification of the data on which they operate and the information they produce. Our approach will be to associate a *semantic record* with each kind of grammar symbol (ID, INTLITERAL, <expression>, and so on). Each different symbol will have a distinct record containing information appropriate for that symbol. Each occurrence of the same kind of symbol will have exactly the same kind of data in its semantic record. Thus an ID can be represented by different kinds of data than an INTLITERAL, but all IDs will have the same format for their semantic records. It is possible for a symbol to have a null semantic record if it requires no semantic data. For example, a semicolon requires no semantic record.

The semantic record for a terminal contains the token's **token_buffer** or some value derived from it. For example, the value of an INTLITERAL, represented as an object of type integer, is derived from the text of the token. Such a record is produced by a semantic routine called just after the parser successfully matches a token against an expected terminal symbol.

The semantic record for a nonterminal is created by a semantic routine that has access to information about any of the symbols on the RHS of the production. If some of these symbols are nonterminals, their corresponding semantic records come from semantic routine calls specified within their own productions. To see how this works, consider

<expression> → <primary> + <primary> #add

A semantic record will be generated for each of the <primary>'s in the RHS of the production. These semantic records record data about each of the operands (for example, where it is stored or what its value is). When **add()** is called, it must be given these records as parameters. It uses them to generate the appropriate code, then produces a new semantic record corresponding to <expression> that records necessary information about the expression just processed.

When a recursive descent parser is used, these semantic records may be stored as local variables of the parsing routines. Records containing the semantic information for nonterminal symbols may be returned as result parameters by corresponding parsing routines. When a table-driven parser is used, an explicit *semantic stack* is typically necessary, as described in Chapter 8, to store semantic records between semantic routine calls.

To define the semantic records, we examine each symbol in the CFG and decide what, if any, semantic information needs to be stored for that symbol. Figure 2.8 shows the semantic records **op_rec** and **expr_rec** required to translate Micro. (**expr_rec** is the first use of an anonymous union, which is not part of standard C.)

All other symbols need no associated semantic information and thus have *null* semantic records. For reasons of efficiency, null records are not explicitly defined and stored anywhere. We can, however, add fields to a semantic record if we decide extra data is needed. In deciding on semantic records, we are really deciding what parameters a semantic routine will have.

```
#define MAXIDLEN        33
typedef char string[MAXIDLEN];

typedef struct operator {     /* for operators */
    enum op { PLUS, MINUS } operator;
} op_rec;

/* expression types */
enum expr { IDEXPR, LITERALEXPR, TEMPEXPR };

/* for <primary> and <expression> */
typedef struct expression {
    enum expr kind;
    union {
        string name;      /* for IDEXPR, TEMPEXPR */
        int val;          /* for LITERALEXPR */
    };
} expr_rec;
```

Figure 2.8 Semantic Records for Micro Grammar Symbols

2.5.5. Action Symbols for Micro

Figure 2.9 illustrates a CFG for Micro that includes action symbols. One production has been added from the previous grammar:

<ident> → ID #process_id

This production is useful because ID appeared in several different contexts in the previous grammar, and we need to call **process_id()** immediately after the parser matches any occurrence of an ID (to access the characters in the **token_buffer** and build an appropriate semantic record). Substituting <ident> for ID everywhere in the grammar is a simple way to ensure that **process_id()** is always called.

We will utilize a few auxiliary routines in our compiler: **generate()** will take four string arguments corresponding to the operation code, two operands, and the result field. It will produce a correctly formated instruction in an output file. **extract()** will take a semantic record and return a string corresponding to the semantic information it contains. This string may be an identifier, an op code, a literal, and so on. The extracted information is fed to **generate()** to create a complete instruction.

Our symbol table routines will be simple because Micro is simple. For example, we need not store any type information as an attribute of an identifier because all identifiers represent integer variables. Because we are generating assembly language instructions that will allow the assembler to allocate storage for variables, we need not record any address information as an

```
<program>          → #start begin <statement list> end
<statement list>   → <statement> {<statement>}
<statement>        → <ident> := <expression> #assign ;
<statement>        → read ( <id list> ) ;
<statement>        → write ( <expr list> ) ;
<id list>          → <ident> #read_id {, <ident> #read_id }
<expr list>        → <expression> #write_expr
                       {, <expression> #write_expr}
<expression>       → <primary>
                       {<add op> <primary> #gen_infix}
<primary>          → ( <expression> )
<primary>          → <ident>
<primary>          → INTLITERAL #process_literal
<add op>           → PLUSOP #process_op
<add op>           → MINUSOP #process_op
<ident>            → ID #process_id
<system goal>      → <program> SCANEOF #finish
```

Figure 2.9 Grammar for Micro with Action Symbols

attribute of an identifier. In fact, no explicit attributes will be used. The only information of interest about an identifier is whether it is already in the symbol table, so the compiler will know if it must generate an instruction that will cause space allocation. The specifications of our symbol table routines are

```
/* Is s in the symbol table? */
extern int lookup(string s);

/* Put s unconditionally into symbol table. */
extern void enter(string s);
```

An auxiliary routine used by a number of the semantic routines is `check_id()`:

```
void check_id(string s)
{
    if (! lookup(s)) {
        enter(s);
        generate("Declare", s, "Integer", "");
    }
}
```

`lookup()` will check whether an entry named `s` is in the symbol table. We will not need to store anything except the name of a symbol in the symbol table. `enter()` will enter string `s` into the symbol table unconditionally. Thus, if necessary, `check_id()` will declare a variable by entering it in the

symbol table and then generating an assembler directive to reserve space for it. In our assembly language, Declare is a pseudo-op that declares a name to the assembler and defines its type. It works for simple, nonstructured global variables. The assembler decides how much space is required for the variable and exactly where it will be allocated.

We also need a routine to allocate temporaries. As mentioned earlier, we will allocate temporaries just like variables. The only difference is that temporary names will be generated by the compiler and are not meaningful in a Micro program. They will have the names Temp&1, Temp&2, and so forth. In a more realistic compiler, temporaries are generally treated as virtual registers, with the code generator having the job of mapping them to real registers, as far as possible. The function `get_temp()` allocates temporaries.

```
char *get_temp(void)
{
    /* max temporary allocated so far */
    static int max_temp = 0;
    static char tempname[MAXIDLEN];

    max_temp++;
    sprintf(tempname, "Temp&%d", max_temp);
    check_id(tempname);
    return tempname;
}
```

We now have the auxiliary routines necessary to define the semantic routines corresponding to Micro's action symbols. They appear in Figure 2.10.

```
void start(void)
{
    /* Semantic initializations, none needed. */
}


void finish(void)
{
    /* Generate code to finish program. */
    generate("Halt", "", "", "");
}


void assign(expr_rec target, expr_rec source)
{
    /* Generate code for assignment. */
    generate("Store", extract(source),
             target.name, "");
}
```

```
op_rec process_op(void)
{
    /* Produce operator descriptor. */
    op_rec o;

    if (current_token == PLUSOP)
        o.operator = PLUS;
    else
        o.operator = MINUS;
    return o;
}

expr_rec gen_infix(expr_rec e1, op_rec op,
                   expr_rec e2)

{
    expr_rec e_rec;
    /* An expr_rec with temp variant set. */
    e_rec.kind = TEMPEXPR;

    /*
     * Generate code for infix operation.
     * Get result temp and set up semantic record
     * for result.
     */
    strcpy(erec.name, get_temp());
    generate(extract(op), extract(e1),
             extract(e2), erec.name);
    return erec;
}

void read_id(expr_rec in_var)
{
    /* Generate code for read. */
    generate("Read", in_var.name,
             "Integer", "");

}

expr_rec process_id(void)
{
    expr_rec t;

    /*
     * Declare ID and build a
     * corresponding semantic record.
     */
    check_id(token_buffer);
    t.kind = IDEXPR;
    strcpy(t.name, token_buffer);
    return t;
}
```

```
expr_rec process_literal(void)
{
    expr_rec t;

    /*
     * Convert literal to a numeric representation
     * and build semantic record.
     */
    t.kind = LITERALEXPR;
    (void) sscanf(token_buffer, "%d", & t.val);
    return t;
}

void write_expr(expr_rec out_expr)
{
    generate("Write", extract(out_expr),
             "Integer", "");
}
```

Figure 2.10 Action Routines for Micro

```
void expression(expr_rec *result)
{
    expr_rec left_operand, right_operand;
    op_rec op;

    primary(& left_operand);
    while (next_token() == PLUSOP ||
           next_token() == MINUSOP) {
        add_op(& op);
        primary(& right_operand);
        left_operand = gen_infix(left_operand, op,
                                 right_operand);
    }
    *result = left_operand;
}
```

Figure 2.11 A Parsing Procedure Including Semantic Processing

Given these semantic routines, we now look again at one of our parsing routines to see how it is altered to handle the inclusion of semantic processing. The procedure **expression()**, shown in Figure 2.11, has been altered to produce an **expr_rec** as an output parameter. Upon return, this **expr_rec** contains the semantic information about the expression recognized by **expression()**. The body of the procedure includes internal variables used to store the semantic records generated by calls to other parsing procedures and

used in a call to the code generation routine `gen_infix()`. (Output parameters in C are accomplished by using *pointers* to the semantic records that are to be affected.)

Example of Recursive Descent Parsing and Translation

As an example, consider the compilation of the following simple Micro program:

begin A := BB – 314 + A ; **end** SCANEOF

Following is a trace of the steps taken by the parser while processing this program, along with the input remaining at each step and the code that would be generated during the processing. The parser actions of interest are calling a parsing routine to find a string in the input to match a nonterminal in the grammar, calling `match()` to match a grammar terminal to an input token, and invoking semantic action routines.

Step	Parser Action	Remaining Input	Generated Code
(1)	Call `system_goal()`	**begin** A:=BB–314+A ; **end** SCANEOF	
(2)	Call `program()`	**begin** A:=BB–314+A ; **end** SCANEOF	
(3)	Semantic Action: `start()`	**begin** A:=BB–314+A ; **end** SCANEOF	
(4)	`match(BEGIN)`	**begin** A:=BB–314+A ; **end** SCANEOF	
(5)	Call `statement_list()`	A:=BB–314+A ; **end** SCANEOF	
(6)	Call `statement()`	A:=BB–314+A ; **end** SCANEOF	
(7)	Call `ident()`	A:=BB–314+A ; **end** SCANEOF	
(8)	`match(ID)`	A:=BB–314+A ; **end** SCANEOF	
(9)	Semantic Action: `process_id()`	:=BB–314+A ; **end** SCANEOF	Declare A,Integer
(10)	`match(ASSIGNOP)`	:=BB–314+A ; **end** SCANEOF	
(11)	Call `expression()`	BB–314+A ; **end** SCANEOF	
(12)	Call `primary()`	BB–314+A ; **end** SCANEOF	
(13)	Call `ident()`	BB–314+A ; **end** SCANEOF	
(14)	`match(ID)`	BB–314+A ; **end** SCANEOF	
(15)	Semantic Action: `process_id()`	–314+A ; **end** SCANEOF	Declare BB,Integer
(16)	Call `add_op()`	–314+A ; **end** SCANEOF	
(17)	`match(MINUSOP)`	–314+A ; **end** SCANEOF	
(18)	Semantic Action: `process_op()`	314+A ; **end** SCANEOF	
(19)	Call `primary()`	314+A ; **end** SCANEOF	
(20)	`match(INTLITERAL)`	314+A ; **end** SCANEOF	
(21)	Semantic Action: `process_literal()`	+A ; **end** SCANEOF	Declare Temp&1,Integer
(22)	Semantic Action: `gen_infix()`	+A ; **end** SCANEOF	Sub BB,314,Temp&1
(23)	Call `add_op()`	+A ; **end** SCANEOF	

(24)	`match(PLUSOP)`	+A ; **end** SCANEOF	
(25)	Semantic Action: `process_op()`	A ; **end** SCANEOF	
(26)	Call `primary()`	A ; **end** SCANEOF	
(27)	Call `ident()`	A ; **end** SCANEOF	
(28)	`match(ID)`	A ; **end** SCANEOF	
(29)	Semantic Action: `process_id()`	; **end** SCANEOF	
(30)	Semantic Action: `gen_infix()`	; **end** SCANEOF	Declare Temp&2,Integer Add Temp&1,A,Temp&2
(31)	Semantic Action: `assign()`	; **end** SCANEOF	Store Temp&2,A
(32)	`match(SEMICOLON)`	; **end** SCANEOF	
(33)	`match(END)`	**end** SCANEOF	
(34)	`match(SCANEOF)`	SCANEOF	
(35)	Semantic Action: `finish()`		Halt

The code generated for

begin A := BB – 314 + A ; **end** SCANEOF

is summarized below. It is easy to verify that it is correct.

```
Declare A,Integer
Declare BB,Integer
Declare Temp&1,Integer
Sub BB,314,Temp&1
Declare Temp&2,Integer
Add Temp&1,A,Temp&2
Store Temp&2,A
Halt
```

Exercises

1. Rewrite the C code in Figure 2.3 using a C **`switch`** statement instead of the sequence of **`if`**s and **`else`**s. Which version do you prefer? Why?
2. Why can't **`EOF`** be used as an enumeration literal instead of **`SCANEOF`**?
3. Implement single character input with either lookahead or pushback in a language without such facilities (e.g., Pascal, Modula-2). Keep in mind the need for both an end-of-line marker and an end-of-file marker.
4. Implement the routines necessary for saving token strings, **`buffer_char()`** and **`clear_buffer()`**, and identifying reserved words, **`check_reserved()`**. Keep the nature of C strings in mind as you do so.

5. The addition of the nonterminal <ident> to the grammar for Micro, as shown in Figure 2.9, will require several changes in the recursive descent parsing routines. Rewrite all the routines that would have to be modified to implement this change.

6. Add code for semantic processing to all parsing routines where it is required, using the `expression()` procedure in Figure 2.11 as a model.

7. A practical parser must have some way of responding to syntax errors encountered in the middle of a parse. The parser for Micro uses recursive descent parsing, requiring that each nonterminal in the grammar have a separate procedure written for it. Using this parsing technique potentially requires that error handling be integrated with each of the parsing procedures. From examination of our parsing procedures for Micro, it is apparent that syntax errors can be detected in either of two ways: `match()` may fail to find the correct token in the source program, or a parsing procedure that examines `next_token()` in a `switch` or `if` statement may fail to find an acceptable token. In the latter case, the Micro parsing routines call `syntax_error()`. `match()` and `syntax_error()` must be designed to take some actions that will allow parsing to continue.

   `match()` could be made into a boolean function in order to indicate whether the required token was found in the input, but such a change would greatly complicate every parsing procedure with any calls to it. A simple alternative would be for `match()` to handle errors by simply pretending that it saw the correct token. In such a case, we must decide if `match()` should consume the incorrect token that it found in the token stream, as it does in the case of a successful match. What are the implications of and the tradeoffs between the two alternative approaches?

   If the second kind of syntax error is encountered, no such simple mechanism is possible, because any of a set of tokens is acceptable to continue the parse. Handling these errors will require explicit reprogramming of some parsing procedures. Suggest a general way that procedures like `statement()` might be changed to handle syntax errors.

8. One optimization that can be implemented even in a compiler as simple as our Micro compiler is *constant folding*, the evaluation of constant expressions at compile-time. If both operands of an expression are constants, there is no need to generate code to evaluate the expression because its value can be determined by the compiler. Identify the semantic action routines that must be changed to implement constant folding, and make the appropriate changes.

9. Suppose you are writing an interpreter for Micro instead of a compiler. The interpreter will execute Micro code as it is being parsed rather than generating assembly language for later execution. How will the semantic action routines and semantic records have to be changed to support interpretation?

10. One useful extension to Micro would be the inclusion of *conditional expressions*, a new form of expression with the following syntax: (E_1 | E_2 |

E_3). When this expression is evaluated, E_2 is returned as the value if E_1 is not zero; otherwise, E_3 is the value of the expression.

(a) Write the appropriate production to add conditional expressions to Micro, including action symbols to specify calls to any required action routines.

(b) Presume the availability of a new assembly language instruction Skip A, which causes the next instruction to be skipped if the operand A evaluates to any value other than 0. Write the action routines necessary to implement conditional expressions.

(c) Don't forget to augment the scanner, too!